Transforming **VIGIL** from a reactive anomaly monitor into a category-defining **Autonomous AI Runtime Immune System** requires solving the exact operational bottlenecks that currently block enterprises from deploying multi-agent pipelines in production.
While **VIGIL v2** effectively catches single-turn volumetric and heuristic spikes (Prompt Injection, Privilege Escalation, 500KB Exfiltration) and enforces binary QUARANTINE, real-world production pipelines fail in subtler ways: indirect cross-agent poisoning, "low-and-slow" chunked exfiltration, tool/MCP server rug-pulls, and self-inflicted downtime from whole-agent quarantines.

🧠 6 Core Product Innovations (Problem → Architecture → Impact)

#	Real-World Production Problem	New VIGIL Feature	How It Works in VIGIL's Stack
1	**Cross-Agent "Confused Deputy" Attacks** *(Agent A reads an untrusted email, summarizes it, and asks trusted Agent B to run a privileged tool. Isolated inspection of Agent B sees a clean internal request.)*	**Cryptographic Taint Tracking & Lineage-Aware Policy Gate (DIFC)**	Stamp every telemetry event with a taint_level (UNTRUSTED_EXT, USER_INPUT, INTERNAL_VERIFIED) and lineage_trace_id. Propagate taint across agent delegations so **ArmorIQ** blocks high-privilege tools whenever upstream lineage is tainted—even if the final prompt looks benign.
2	**Binary Quarantine Kills Uptime** *(Quarantining an entire agent like CustomerBot on one attack or false positive takes down*	**Surgical Capability Downgrading & Shadow Honeypot Routing ("Mirage Mode")**	Replace binary QUARANTINE with graduated containment: **(a) Session-Scoped Tool Stripping** (revoke write/admin tools for the

	the service for all legitimate users—creating a self-inflicted DoS.)		offending trace while keeping the agent live) and **(b) Shadow Sandbox Routing** (silently swap real SQLite/API connections to a synthetic honey-twin so the attacker thinks they succeeded while VIGIL records their full kill chain).
3	**"Low-and-Slow" Salami Exfiltration** *(Attackers bypass EWMA and Isolation Forest by extracting 2KB across 250 turns instead of a single 500KB spike, or splitting requests across multiple sub-agents.)*	**Cumulative Entropy Ledgers & Canary Tripwires**	Maintain a sliding-window **Exfiltration Budget Ledger** per principal/trace tracking cumulative bytes, Shannon entropy, and sensitive record counts across turns. Inject invisible cryptographic **Canary Tokens** (synthetic honey-rows) into database responses; if any agent emits a canary hash, trigger an instant 100% deterministic containment.
4	**MCP & Tool Output Poisoning**	**Bidirectional MCP Firewall & Schema Hash Pinning**	Pin SHA-256 hashes of all registered tool

	(Agents dynamically call external MCP tools that return hidden prompt injections inside JSON responses, or silently mutate their tool schemas after registration.)		schemas inside VIGIL's hash-chained SQLite store to block mid-runtime "rug-pulls." Scan **tool return payloads** (ingress from tools) before they enter the agent's context window, neutralizing indirect prompt injections before the LLM processes them.
5	**Static Detection Rules Decay Fast** *(In v2, Claude explains the incident to humans, but Layer 1 heuristics stay static until an engineer manually writes a new rule.)*	**Closed-Loop "Adaptive Immunity" (Auto-Antibody Synthesis)**	Upgrade the **Claude AI Incident Explainer** into an **Active Antibody Generator**. When Layer 3 (Isolation Forest) catches a zero-day behavioral anomaly, Claude synthesizes a deterministic Layer 1 heuristic/ArmorIQ rule, automatically backtests it against /backend/storage/ replay logs to verify 0% false-positive regression, and stages it for 1-click deployment.
6	**Lingering Blast Radius Before Containment**	**Transactional Intent Checkpoints & Compensating**	Register a compensating rollback hook

	(If an agent executes two tool calls and gets quarantined on the third, the first two side effects remain active.)	**Rollbacks**	(undo_action) alongside every state-mutating request at POST /intent/verify. When a trace is quarantined, the **Remediation Executor** walks backward through the hash-chained audit trail to revoke tokens, revert staging writes, and visualize the exact **Blast Radius DAG** in React.

🏗️ Upgraded System Architecture (VIGIL v3 Immune Loop)





💻 Concrete Implementation Blueprints for Your Codebase

1. Cross-Agent Taint Propagation & Lineage Gate (/backend/services/taint_tracker.py)

Solve the "Confused Deputy" problem by tracking whether an agent's current context has been contaminated by untrusted data upstream before allowing privileged delegations or tool calls:

```python
from enum import IntEnum
from dataclasses import dataclass, field
```

```
from typing import Dict, List, Set

class TaintLevel(IntEnum):
    SYSTEM_TRUSTED = 0      # Internal system prompts & signed configs
    VERIFIED_INTERNAL = 1   # Clean internal agent state
    USER_UNTRUSTED = 2      # Direct external user input
    EXTERNAL_POISON_RISK = 3 # Unverified MCP output, web scrape, external email

@dataclass
class TraceLineage:
    trace_id: str
    current_taint: TaintLevel = TaintLevel.SYSTEM_TRUSTED
    agent_hops: List[str] = field(default_factory=list)
    taint_sources: Set[str] = field(default_factory=set)

class ProvenanceEngine:
    def __init__(self):
        self._traces: Dict[str, TraceLineage] = {}
        # Tools that NEVER accept taint >= USER_UNTRUSTED without explicit declassification
        self.restricted_sinks = {
            "billing_database": TaintLevel.VERIFIED_INTERNAL,
            "export_financial_records": TaintLevel.SYSTEM_TRUSTED,
            "iam_role_grant": TaintLevel.SYSTEM_TRUSTED,
        }

    def propagate_delegation(
        self, trace_id: str, from_agent: str, to_agent: str, payload_taint: TaintLevel, source_label: str
    ) -> TraceLineage:
        lineage = self._traces.get(trace_id, TraceLineage(trace_id=trace_id))
        # Taint is monotonically non-decreasing across agent hops
        if payload_taint >= lineage.current_taint:
            lineage.current_taint = payload_taint
            lineage.taint_sources.add(source_label)
        lineage.agent_hops.append(f"{from_agent}->{to_agent}")
        self._traces[trace_id] = lineage
        return lineage

    def evaluate_sink_policy(self, trace_id: str, tool_name: str) -> dict:
        lineage = self._traces.get(trace_id)
        if not lineage:
            return {"allowed": False, "reason": "Missing cryptographic lineage trace (Fail-Closed)"}

        # Detect circular delegation loops (Denial-of-Wallet protection)
```

```
        if len(lineage.agent_hops) > 6:
            return {"allowed": False, "reason": f"Delegation depth exceeded ({len(lineage.agent_hops)} hops)"}

        max_allowed_taint = self.restricted_sinks.get(tool_name, TaintLevel.EXTERNAL_POISON_RISK)
        if lineage.current_taint > max_allowed_taint:
            return {
                "allowed": False,
                "reason": (
                    f"DIFC Taint Violation: Tool '{tool_name}' requires taint <= {max_allowed_taint.name}, "
                    f"but trace carries {lineage.current_taint.name} from {list(lineage.taint_sources)}"
                ),
            }
        return {"allowed": True, "taint_level": lineage.current_taint.name}
```

2. Graduated Containment: Surgical Tool Stripping & Mirage Honeypot (/backend/services/remediation.py)

Upgrade your remediation executor from a blunt kill-switch to a 4-tier immune response that preserves business continuity while trapping attackers:

```
Python
from enum import Enum

class ContainmentAction(str, Enum):
    LOG_ONLY = "LOG_ONLY"                   # Score < 40%: Audit only
    SURGICAL_DOWNGRADE = "DOWNGRADE_SCOPE"  # Score 40-74%: Strip write/admin tools for this session only
    MIRAGE_HONEYPOT = "SHADOW_SANDBOX"      # Score 75-89%: Route attacker to synthetic DB twin to harvest TTPs
    FULL_QUARANTINE = "QUARANTINE_ROLLBACK" # Score >= 90% or Canary Tripwire: Freeze agent + run rollback hooks

def select_adaptive_remediation(threat_score: float, canary_triggered: bool, is_critical_agent: bool) -> dict:
    if canary_triggered or threat_score >= 0.92:
        return {
            "mode": ContainmentAction.FULL_QUARANTINE,
            "execute_compensating_rollbacks": True
```

```
            "summary": "Deterministic breach or critical severity: quarantining agent and rolling back session side-effects."
        }
    if 0.75 <= threat_score < 0.92:
        return {
            "mode": ContainmentAction.MIRAGE_HONEYPOT,
            "target_datasource": "sqlite+aiosqlite:///./backend/storage/honeypot_mirage.db",
            "summary": "High-confidence attack redirected to Mirage Honeypot to capture attacker objectives without downtime."
        }
    if 0.45 <= threat_score < 0.75 or is_critical_agent:
        return {
            "mode": ContainmentAction.SURGICAL_DOWNGRADE,
            "revoked_capabilities": ["db_write", "external_http_post", "agent_delegate"],
            "retained_capabilities": ["faq_search", "read_public_docs"],
            "summary": "Elevated risk: stripped state-mutating capabilities for session while keeping agent online."
        }
    return {"mode": ContainmentAction.LOG_ONLY, "summary": "Within normal behavioral envelope."}
```

3. Closed-Loop "Auto-Vaccination" Engine (/backend/services/antibody_synthesizer.py)

Give the words **"Behavioral Immune System"** literal engineering truth. When an attack is caught, use Claude to generate a candidate rule and automatically test it against your stored benign telemetry before promoting it:

1. **Capture**: An incident triggers via Isolation Forest (Layer 3) or Canary Tripwire.
2. **Synthesize**: Claude generates both the **3-sentence operator explanation** AND a structured JSON **Candidate Antibody** (a fast Layer 1 pattern/threshold rule + ArmorIQ intent constraint).
3. **Replay Backtest**: VIGIL runs the candidate rule against the last 500 normal telemetry events in /backend/storage/ to compute a **False Positive Rate (FPR)**.
4. **Hot-Reload**: If FPR == 0.0% and detection recall on the attack payload is 100%, the dashboard displays a **"⚡ Deploy Verified Antibody (0% FPR)"** button on the incident card.

🧪 2 New High-Impact Demo Scenarios to Win Judges & Customers

Add these two buttons to your bottom demo panel right next to your existing Scenarios 1–4. They demonstrate capabilities that almost no other agent security project has:

🔥 Scenario 5: Indirect Cross-Agent Taint Attack ("The Confused Deputy")

- **The Attack**: CustomerBot reads a support ticket containing hidden instructions. CustomerBot does *not* attack the database itself—instead, it sends a polite, normal-looking summary to AnalyticsAgent, which then asks BillingAgent to export records.
- **What to Show on the Dashboard**:
  - Individual payload heuristics on BillingAgent score only **22% (Benign Text)**—showing why standard guardrails fail.
  - However, VIGIL's **Lineage Taint Tracker** lights up red across the 3-node graph (CustomerBot → AnalyticsAgent → BillingAgent), flagging TAINT: EXTERNAL_POISON_RISK.
  - **ArmorIQ Policy Gate** blocks the action based on **Lineage Provenance**, proving VIGIL catches multi-hop attacks that bypass single-agent prompt filters.

🔥 Scenario 6: "Low-and-Slow" Exfiltration + Mirage Honeypot Redirection

- **The Attack**: Compromised AnalyticsAgent attempts to steal customer records in tiny 1.8KB chunks to stay under the EWMA single-request volume threshold, hitting a seeded **Canary Record** on chunk #4.
- **What to Show on the Dashboard**:
  - The **Cumulative Entropy Ledger** climbs steadily across chunks 1–3, and hits 100% CRITICAL the millisecond the **Canary Tripwire** is touched on chunk #4.
  - Instead of throwing a 403 error, VIGIL activates **Mirage Mode (amber pulsing badge)**: the attacker is transparently switched to a synthetic decoy database while legitimate queries to AnalyticsAgent continue uninterrupted.
  - Within 3 seconds, the **Antibody Synthesizer** badge pops up: *"New Layer-1 Antibody Verified against 500 historical logs (0.0% False Positives) — Click to Immunize Fleet."*

🎨 Frontend Dashboard UI Upgrades (React 18 + Zustand + Tailwind)

To make these backend innovations immediately visual within 5 seconds of looking at the screen:

- **Live Agent Topology & Taint Graph (Top Center Panel)**: Replace static agent cards with a connected 3-node graph (CustomerBot ⇄ AnalyticsAgent ⇄ BillingAgent). Animate glowing packet dots moving between agents—green for verified internal calls, amber for untrusted user taint, and crimson for blocked delegations.
- **Containment Spectrum Pill**: Upgrade the agent status badge from binary NORMAL / QUARANTINED to a 4-state tactical badge:
  - ● NORMAL (Emerald)
  - ◐ SCOPED DOWNGRADE (Cyan — *Write tools stripped, Read active*)
  - ◈ MIRAGE HONEYPOT (Amber — *Attacker routed to synthetic decoy DB*)
  - ■ QUARANTINED + ROLLED BACK (Crimson pulse)
- **One-Click "Immunize Fleet" Action in Threat Overlay**: Inside the slide-over detail drawer (where you currently show the Rule Engine (60%) contribution), add an

Auto-Synthesized Antibody card showing the replay test score (Backtested on 500 benign events: 0 False Positives) with an interactive **Deploy Rule to Layer 1** button.

Interactive Architecture Explorer: VIGIL v3 Adaptive Immune Flow

Step through the decision tree below to inspect how incoming agent telemetry and multi-hop delegations move through taint tracking, 3-layer detection, canary verification, and graduated ArmorIQ containment: